J MUKHESH

2303A51813

Delete and earn

```
class Solution {
   public int deleteAndEarn(int[] nums) {
      int max = 0;
      for(int val : nums)
         max = Math.max(max,val);
      int[] dp = new int[max + 1];
      for(int val : nums){
         dp[val] += val;
      }
      for(int i = 2; i < dp.length; i++){
         dp[i] = Math.max(dp[i] + dp[i - 2],dp[i - 1]);
      }
      return dp[max];
   }
}
```

Partition equal subset sum

```
class Solution {
   public boolean rec(int[] nums,int target,Boolean[][] dp,int n){
      if(target == 0)
         return true;
      if(n == 0){
         if(target == nums[0])
            return true;
         return false;
      }
```

```
        if(dp[n][target] != null)
            return dp[n][target];
        boolean pick = false;
        if(target >= nums[n])
            pick = rec(nums,target - nums[n],dp,n - 1);
        boolean notPick = rec(nums,target,dp,n - 1);
        return dp[n][target] = pick || notPick;
    }
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for(int val : nums)
            sum += val;
        if(sum % 2 != 0)
            return false;
        Boolean[][] dp = new Boolean[nums.length][sum/2 + 1];
        return rec(nums,sum / 2,dp,nums.length - 1);
    }
}
```

Target sum

```
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = 0;
        for(int val : nums)
            sum += val;
        if(sum < Math.abs(target) || (sum + target) % 2 != 0)
            return 0;
        int pos = (sum + target)/2;
        int[] dp = new int[pos + 1];
```

```
        dp[0] = 1;
        for (int num : nums) {
            for (int s = pos; s >= num; s--) {
                dp[s] += dp[s - num];
            }
        }
        return dp[pos];
    }
}
```